



RTL DESIGN LAB FILE

SUBMITTED BY:

NAME	ABUBAKAR ATIQUE
CNIC	42301-3233797-1

SUBMITTED TO:

Engr. Khalil Ur Rehman

INDEX

Week # 1 Tasks:	3
AND Gate:	3
OR Gate:	3
XOR Gate:	4
Half Adder:	4
Full Adder / Full adder using two Half Adders:	5
Combinational Logic:	6
Binary Adder/Subtractor:	6
MUX:	7
BCD to 7 segment:	7
Priority Encoder:	8
Decoder:	8
Multiplier:	9
Week # 2 Tasks:	10
D-Flipflop:	10
T-Flipflop:	10
SR-Flipflop:	11
JK-Flipflop:	11
Conversion of T-FF into D-FF:	12
Synchronous Up/Down Counter:	12
Asynchronous Up/Down Counter:	13
BCD Counter:	14
Mod-N Counter:	14
Frequency Divider 100Mhz-25Mhz:	15
PISO Register:	15
Bidirectional Shift Register:	16
Universal Shift Register:	16
Ring Counter:	17
Jhonsen Counter:	18
Multiplier using Pipelining:	18
ALU using Pipelining:	19



Week # 1 Tasks:

AND Gate:

Design:

```

1 //Abubakar_Atique_Duet//
2 module and_behavioral (
3     input a, b,
4     output reg y
5 );
6     always @(*) begin
7         y = a & b;
8     end
9 endmodule
10
11 module and_dataflow (
12     input a, b,
13     output y
14 );
15     assign y = a & b;
16 endmodule
17
18 module and_structural (
19     input a, b,
20     output y
21 );
22     and g1 (y, a, b);
23 endmodule

```

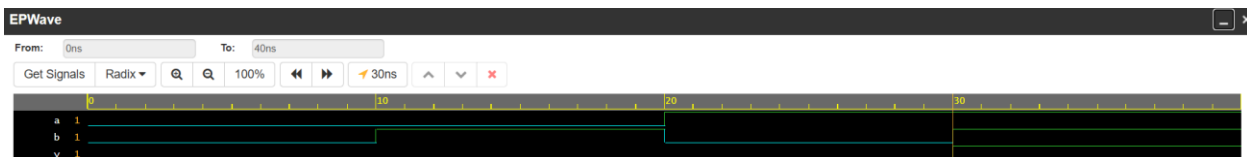
TestBench

```

1 //Abubakar_Atique_Duet//
2 module tb_and;
3     reg a, b;
4     wire y;
5
6     and_dataflow uut (.a(a),.b(b),.y(y));
7
8     initial begin
9         $dumpfile("dump.vcd");
10        $dumpvars(0);
11        a = 0; b = 0; #10;
12        a = 0; b = 1; #10;
13        a = 1; b = 0; #10;
14        a = 1; b = 1; #10;
15        $finish;
16    end
17 endmodule

```

EP Wave:



OR Gate:

Design:

```

1 //Abubakar_Atique_Duet//
2 module or_behavioral (
3     input a, b,
4     output reg y
5 );
6     always @(*) begin
7         y = a | b;
8     end
9 endmodule
10
11 module or_dataflow (
12     input a, b,
13     output y
14 );
15     assign y = a | b;
16 endmodule
17
18 module or_structural (
19     input a, b,
20     output y
21 );
22     or g1 (y, a, b);
23 endmodule
24

```

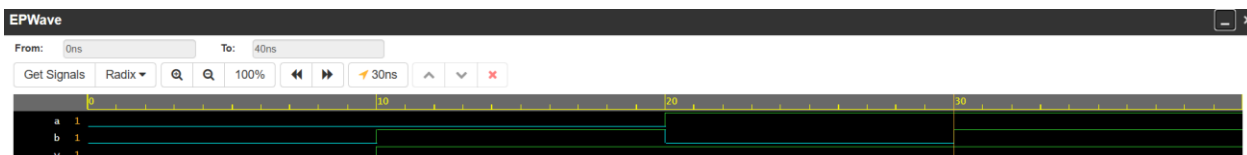
TestBench:

```

1 //Abubakar_Atique_Duet//
2 module tb_or;
3     reg a, b;
4     wire y;
5
6     or_dataflow uut (.a(a),.b(b),.y(y));
7
8     initial begin
9         $dumpfile("dump.vcd");
10        $dumpvars(1, tb_or);
11
12        a = 0; b = 0; #10;
13        a = 0; b = 1; #10;
14        a = 1; b = 0; #10;
15        a = 1; b = 1; #10;
16        $finish;
17    end
18 endmodule
19

```

EP Wave:





XOR Gate:

Design:

```

1 //Abubakar_Atique_Duet//
2 module xor_behavioral (
3     input a, b,
4     output reg y
5 );
6     always @(*) begin
7         y = a ^ b;
8     end
9 endmodule
10
11 module xor_dataflow (
12     input a, b,
13     output y
14 );
15     assign y = a ^ b;
16 endmodule
17
18 module xor_structural (
19     input a, b,
20     output y
21 );
22     xor g1 (y, a, b);
23 endmodule

```

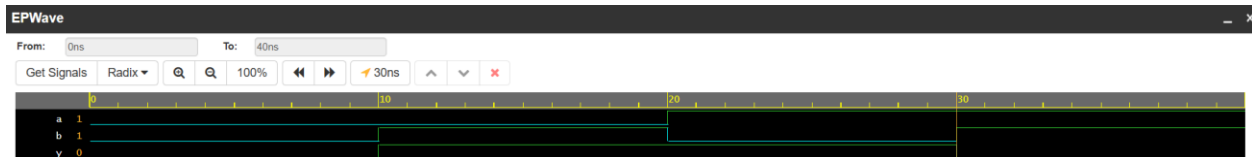
TestBench:

```

1 //Abubakar_Atique_Duet//
2 module tb_xor;
3     reg a, b;
4     wire y;
5
6     xor_dataflow uut (.a(a),.b(b),.y(y));
7
8     initial begin
9         $dumpfile("dump.vcd");
10        $dumpvars 0;
11
12        a = 0; b = 0; #10;
13        a = 0; b = 1; #10;
14        a = 1; b = 0; #10;
15        a = 1; b = 1; #10;
16        $finish;
17    end
18 endmodule
19

```

EP Wave:



Half Adder:

Design:

```

1 // Code your design here
2 module adder (
3     input logic a,
4     input logic b,
5     output logic sum,
6     output logic cout
7 );
8     assign sum=a^b;
9     assign cout=a&b;
10
11 endmodule

```

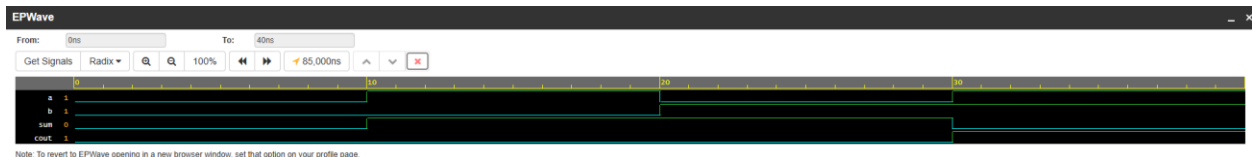
TestBench:

```

1 // Code your testbench here
2 // or browse Examples
3 module tb;
4     logic a,b,sum,cout;
5
6     adder dut(.a(a),.b(b),.sum(sum),.cout(cout));
7
8     initial begin
9         a=0;b=0;#10;
10
11        a=1;b=0;#10;
12        a=0;b=1;#10;
13        a=1;b=1;#10;
14        $finish;
15    end
16    initial begin
17        $dumpvcd("dump.vcd");
18        $dumpvars(0);
19    end
20 endmodule

```

EP Wave:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

Full Adder / Full adder using two Half Adders:

Design:

```

1 //Abubakar_Atique_Duet//
2 module full_adder(
3     input logic a,
4     input logic b,
5     input logic cin,
6     output logic sum,
7     output logic cout
8 );
9     assign sum=a^b^cin;
10    assign cout=(a&b)|(b&cin)|(a&cin);
11
12 endmodule
13
14 module half_adder(
15     input logic a,
16     input logic b,
17     output logic sum,
18     output logic cout
19 );
20     assign sum=a^b;
21     assign cout=a&b;
22
23 endmodule
24
25 module full_adder_using_ha(
26     input logic a,
27     input logic b,
28     input logic cin,
29     output logic sum,
30     output logic cout
31 );
32
33 wire s1,c1,c2;
34
35 half_adder ha1(.a(a),.b(b),.sum(s1),.cout(c1));
36 half_adder ha2(.a(s1),.b(cin),.sum(sum),.cout(c2));
37
38 assign cout=c1|c2;
39
40 endmodule

```

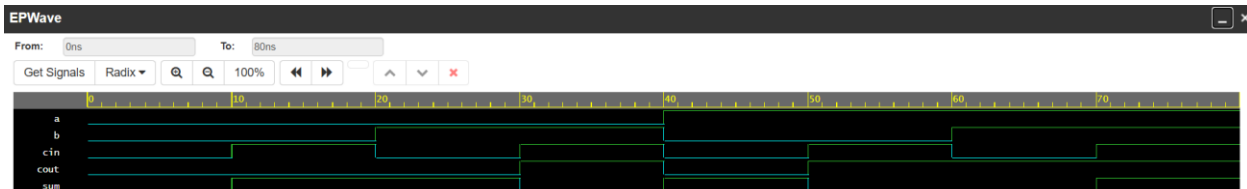
TestBench:

```

1 //Abubakar_Atique_Duet//
2 module tb;
3     logic a,b,cin,sum,cout;
4
5     full_adder_using_ha dut(.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));
6
7     initial begin
8         a=0;b=0;cin=0;#10;
9         a=0;b=0;cin=1;#10;
10        a=0;b=1;cin=0;#10;
11        a=0;b=1;cin=1;#10;
12        a=1;b=0;cin=0;#10;
13        a=1;b=0;cin=1;#10;
14        a=1;b=1;cin=0;#10;
15        a=1;b=1;cin=1;#10;
16        $finish;
17    end
18    initial begin
19        $dumpvcd("dump.vcd");
20        $dumpvars(0);
21    end
22 endmodule

```

EP Wave:



Combinational Logic:

Design:

```

1 //*****Abubakar Atique DUET*****//
2 module combinational_logic (
3     input  logic A, B, C,
4     output logic F1, F2
5 );
6
7     logic T1, T2, T3;
8     logic F2_not;
9
10    assign F2 = (A & B) | (A & C) | (B & C);
11    assign F2_not = ~F2;
12
13    assign T1 = A | B | C;
14    assign T2 = A & B & C;
15
16    assign T3 = T1 & F2_not;
17
18    assign F1 = T2 | T3;
19
20 endmodule

```

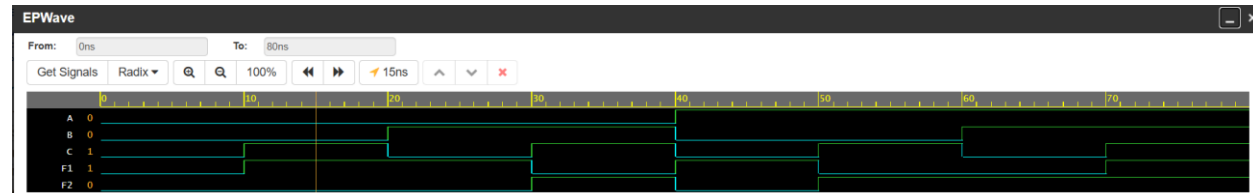
TestBench:

```

1 //*****Abubakar Atique DUET*****//
2 module tb_combinational_logic;
3
4     logic A, B, C;
5     logic F1, F2;
6
7     combinational_logic dut (
8         .A(A),
9         .B(B),
10        .C(C),
11        .F1(F1),
12        .F2(F2)
13    );
14
15    initial begin
16        $dumpfile("dump.vcd");
17        $dumpvars();
18
19        A = 0; B = 0; C = 0; #10;
20        A = 0; B = 0; C = 1; #10;
21        A = 0; B = 1; C = 0; #10;
22        A = 0; B = 1; C = 1; #10;
23        A = 1; B = 0; C = 0; #10;
24        A = 1; B = 0; C = 1; #10;
25        A = 1; B = 1; C = 0; #10;
26        A = 1; B = 1; C = 1; #10;
27
28        $finish;
29    end
30
31 endmodule

```

EP Wave:



Binary Adder/Subtractor:

Design:

```

1 //Abubakar Atique Duet//
2 module full_adder (
3     input  logic a, b, cin,
4     output logic sum, cout
5 );
6
7     assign sum = a ^ b ^ cin;
8     assign cout = (a & b) | (cin & (a ^ b));
9 endmodule
10
11 module adder_subtractor (
12     input  logic [3:0] x,y,
13     input  logic M,
14     output logic [3:0] F,
15     output logic Cy
16 );
17
18     logic c1, c2, c3;
19
20     logic [3:0] b_xor;
21     assign b_xor[0] = y[0] ^ M;
22     assign b_xor[1] = y[1] ^ M;
23     assign b_xor[2] = y[2] ^ M;
24     assign b_xor[3] = y[3] ^ M;
25
26     full_adder fa0 (.a(x[0]), .b(b_xor[0]), .cin(M), .sum(F[0]), .cout(c1));
27     full_adder fa1 (.a(x[1]), .b(b_xor[1]), .cin(c1), .sum(F[1]), .cout(c2));
28     full_adder fa2 (.a(x[2]), .b(b_xor[2]), .cin(c2), .sum(F[2]), .cout(c3));
29     full_adder fa3 (.a(x[3]), .b(b_xor[3]), .cin(c3), .sum(F[3]), .cout(cy));
30 endmodule

```

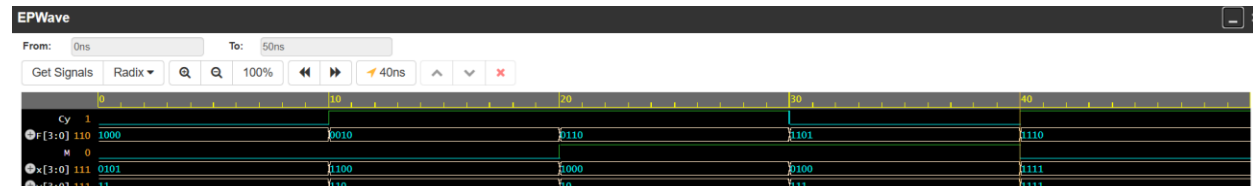
TestBench:

```

1 //Abubakar Atique Duet//
2 module tb_adder_subtractor;
3     logic [3:0] x,y,F;
4     logic M,Cy;
5
6     adder_subtractor dut (.x(x),.y(y),.M(M),.F(F),.Cy(Cy));
7
8     initial begin
9         $dumpfile("dump.vcd");
10        $dumpvars(0, tb_adder_subtractor);
11
12        x = 4'd5; y = 4'd3; M = 1'b0; #10;
13        x = 4'd12; y = 4'd6; M = 1'b0; #10;
14        x = 4'd8; y = 4'd2; M = 1'b1; #10;
15        x = 4'd4; y = 4'd7; M = 1'b1; #10;
16        x = 4'd15; y = 4'd15; M = 1'b0; #10;
17
18        $finish;
19    end
20 endmodule
21

```

EP Wave:



MUX:

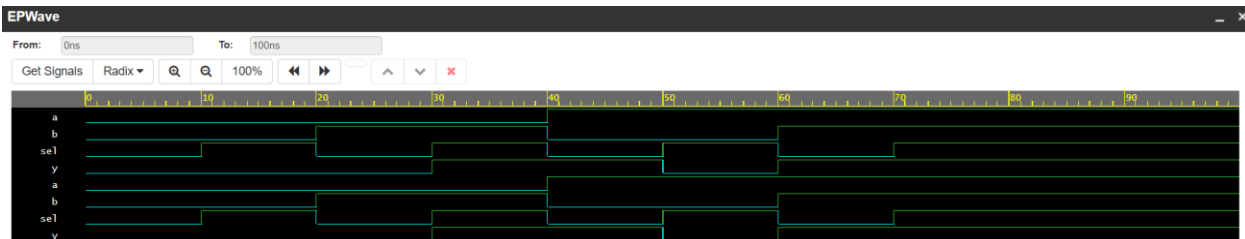
Design:

```
1 /*****Abubakar Atique DUET*****/
2 module mux(
3     input logic a,b,sel,
4     output logic y
5 );
6     assign y=(a & ~sel)|(b & sel);
7 endmodule
```

TestBench:

```
1 /*****Abubakar Atique DUET*****/
2 module tb_mux;
3     logic a,b,sel,y;
4     mux dut(.a(a),.b(b),.sel(sel),.y(y));
5     initial begin
6         $dumpfile("dump.vcd");
7         $dumpvars();
8
9         a=0;b=0;sel=0;#10;
10        a=0;b=0;sel=1;#10;
11        a=0;b=1;sel=0;#10;
12        a=0;b=1;sel=1;#10;
13        a=1;b=0;sel=0;#10;
14        a=1;b=0;sel=1;#10;
15        a=1;b=1;sel=0;#10;
16        a=1;b=1;sel=1;#10;
17
18        #20;
19        $finish;
20    end
21 endmodule
```

EP Wave:



BCD to 7 segment:

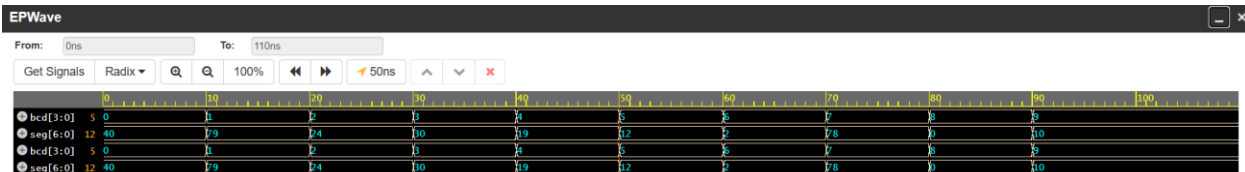
Design:

```
1 /*****Abubakar Atique DUET*****/
2 module bcd_to_7seg (
3     input logic [3:0] bcd,
4     output logic [6:0] seg
5 );
6
7     always_comb begin
8         case (bcd)
9             4'h0: seg = 7'b100_0000;
10            4'h1: seg = 7'b111_1001;
11            4'h2: seg = 7'b010_0100;
12            4'h3: seg = 7'b011_0000;
13            4'h4: seg = 7'b001_1001;
14            4'h5: seg = 7'b001_0010;
15            4'h6: seg = 7'b000_0010;
16            4'h7: seg = 7'b111_1000;
17            4'h8: seg = 7'b000_0000;
18            4'h9: seg = 7'b001_0000;
19            default: seg = 7'b111_1111;
20        endcase
21    end
22 endmodule
```

TestBench:

```
1 /*****Abubakar Atique DUET*****/
2 module bcd_to_7seg_tb;
3
4     logic [3:0] bcd;
5     logic [6:0] seg;
6
7     bcd_to_7seg dut (.bcd(bcd),.seg(seg));
8
9     initial begin
10        $dumpfile("dump.vcd");
11        $dumpvars(0, bcd_to_7seg_tb);
12
13        for (int i = 0; i <= 9; i++) begin
14            bcd = i;
15            #10;
16        end
17        #10;
18        $finish;
19    end
20 endmodule
```

EP Wave:



Priority Encoder:

Design:

```

1  /***** Abubakar Atique DUET *****/
2  module priority_encoder (
3      input logic [3:0] I,
4      output logic [1:0] Y,
5      output logic V
6  );
7
8      always_comb begin
9          casez (I)
10             4'b1??? : begin Y = 2'b11; V = 1'b1; end
11             4'b01?? : begin Y = 2'b10; V = 1'b1; end
12             4'b001? : begin Y = 2'b01; V = 1'b1; end
13             4'b0001 : begin Y = 2'b00; V = 1'b1; end
14             4'b0000 : begin Y = 2'bxx; V = 1'b0; end
15             default : begin Y = 2'bxx; V = 1'b0; end
16          endcase
17      end
18  endmodule

```

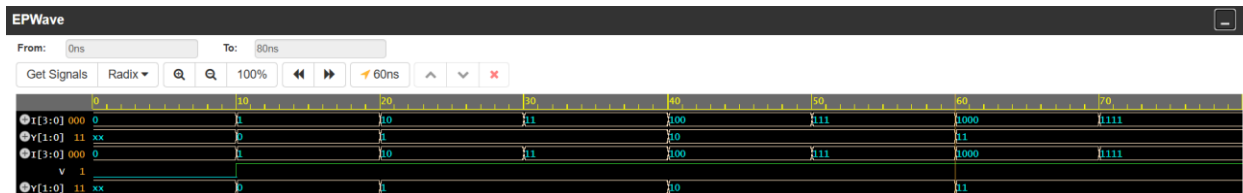
TestBench:

```

1  /***** Abubakar Atique DUET *****/
2  module tb_priority_encoder;
3      logic [3:0] I;
4      logic [1:0] Y;
5
6      priority_encoder dut (.I(I),.Y(Y));
7
8      initial begin
9          $dumpfile("dump.vcd");
10         $dumpvars(0, tb_priority_encoder);
11
12         I = 4'b0000; #10;
13         I = 4'b0001; #10;
14         I = 4'b0010; #10;
15         I = 4'b0011; #10;
16         I = 4'b0100; #10;
17         I = 4'b0111; #10;
18         I = 4'b1000; #10;
19         I = 4'b1111; #10;
20         $finish;
21     end
22 endmodule

```

EP Wave:



Decoder:

Design:

```

1  /***** Abubakar Atique DUET *****/
2  module decoder_2to4 (
3      input logic [1:0] A, // [A1, A0]
4      input logic E, // Enable
5      output logic [3:0] Y // [Y3, Y2, Y1, Y0]
6  );
7
8      always_comb begin
9          if (!E) Y = 4'b0000;
10         else begin
11             case (A)
12                 2'b00: Y = 4'b0001;
13                 2'b01: Y = 4'b0010;
14                 2'b10: Y = 4'b0100;
15                 2'b11: Y = 4'b1000;
16                 default: Y = 4'b0000;
17             endcase
18         end
19     end
20 endmodule

```

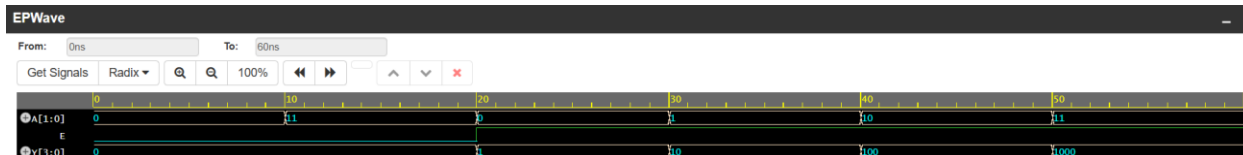
TestBench:

```

1  /***** Abubakar Atique DUET *****/
2  module tb_decoder_2to4;
3      logic [1:0] A;
4      logic E;
5      logic [3:0] Y;
6
7      decoder_2to4 uut (.A(A),.E(E),.Y(Y));
8
9      initial begin
10         $dumpfile("dump.vcd");
11         $dumpvars(0, tb_decoder_2to4);
12
13         // Disabled state
14         E = 1'b0; A = 2'b00; #10;
15         E = 1'b0; A = 2'b11; #10;
16         // Enabled states
17         E = 1'b1; A = 2'b00; #10;
18         E = 1'b1; A = 2'b01; #10;
19         E = 1'b1; A = 2'b10; #10;
20         E = 1'b1; A = 2'b11; #10;
21
22         $finish;
23     end
24 endmodule

```

EP Wave:





Multiplier:

Design:

```

1  /***** Abubakar Atique DUET *****/
2  module multiplier_4x2 (
3      input logic [3:0] A,
4      input logic [1:0] B,
5      output logic [5:0] P
6  );
7
8      logic [3:0] pp0; // Row 0 partial product bits: A * B[0]
9      logic [3:0] pp1; // Row 1 partial product bits: A * B[1] (shifted left)
10     logic [2:0] c; // Ripple carry lines between adders
11
12     always_comb begin
13         // 1. Partial Product Generation using 8 AND gates
14         pp0 = A & {4{B[0]}}; // [A3&B0, A2&B0, A1&B0, A0&B0]
15         pp1 = A & {4{B[1]}}; // [A3&B1, A2&B1, A1&B1, A0&B1]
16
17         // 2. Addition Logic matching Row 0 + Row 1 (shifted by 1 bit)
18         P[0] = pp0[0]; // First bit drops straight down
19
20         // Half Adder 1: Adds pp0[1] and pp1[0]
21         P[1] = pp0[1] ^ pp1[0];
22         c[0] = pp0[1] & pp1[0];
23
24         // Full Adder 1: Adds pp0[2], pp1[1], and carry from previous stage
25         P[2] = pp0[2] ^ pp1[1] ^ c[0];
26         c[1] = (pp0[2] & pp1[1]) | (c[0] & (pp0[2] ^ pp1[1]));
27
28         // Full Adder 2: Adds pp0[3], pp1[2], and carry from previous stage
29         P[3] = pp0[3] ^ pp1[2] ^ c[1];
30         c[2] = (pp0[3] & pp1[2]) | (c[1] & (pp0[3] ^ pp1[2]));
31
32         // Half Adder 2: Adds the final remaining bit pp1[3] and the last
33         carry
34         P[4] = pp1[3] ^ c[2];
35         P[5] = pp1[3] & c[2]; // Final carry becomes the most significant bit
36     end
37 endmodule

```

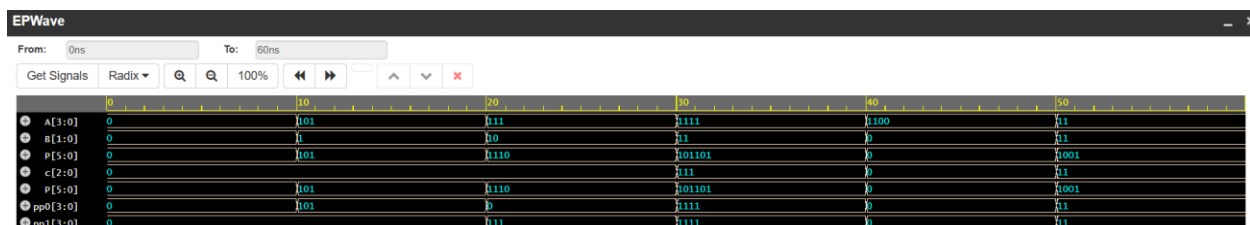
TestBench:

```

1  /***** Abubakar Atique DUET *****/
2  module tb_multiplier_4x2;
3      logic [3:0] A;
4      logic [1:0] B;
5      logic [5:0] P;
6
7      multiplier_4x2 uut (
8          .A(A),
9          .B(B),
10         .P(P)
11     );
12
13     initial begin
14         $dumpfile("dump.vcd");
15         $dumpvars(0, tb_multiplier_4x2);
16
17         // Test 0: 0 * 0 = 0
18         A = 4'd0; B = 2'd0; #10;
19
20         // Test 1: 5 * 1 = 5
21         A = 4'd5; B = 2'd1; #10;
22
23         // Test 2: 7 * 2 = 14
24         A = 4'd7; B = 2'd2; #10;
25
26         // Test 3: 15 * 3 = 45 (Maximum values)
27         A = 4'd15; B = 2'd3; #10;
28
29         // Test 4: 12 * 0 = 0
30         A = 4'd12; B = 2'd0; #10;
31
32         // Test 5: 3 * 3 = 9
33         A = 4'd3; B = 2'd3; #10;
34
35         $finish;
36     end
37 endmodule

```

EP Wave:



Week # 2 Tasks:

D-Flipflop:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module d_ff (
3      input logic clk, rst, d,
4      output logic q
5  );
6      always_ff @(posedge clk or posedge rst) begin
7          if (rst) q <= 1'b0;
8          else q <= d;
9      end
10 endmodule
11

```

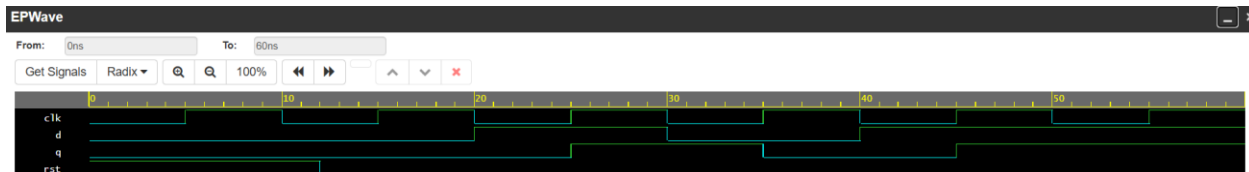
TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module tb_d_ff;
3      logic clk, rst, d, q;
4
5      d_ff dut (.clk(clk), .rst(rst), .d(d), .q(q));
6
7      always #5 clk = ~clk;
8
9      initial begin
10         $dumpfile("dump.vcd"); $dumpvars(0, tb_d_ff);
11
12         clk = 0; rst = 1; d = 0;
13         #12 rst = 0; // Release reset
14         #8 d = 1; // q becomes 1
15         #10 d = 0; // q becomes 0
16         #10 d = 1; // q becomes 1
17         #20 $finish;
18     end
19 endmodule

```

EP Wave:



T-Flipflop:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module t_ff (
3      input logic clk, rst, t,
4      output logic q
5  );
6      always_ff @(posedge clk or posedge rst) begin
7          if (rst) q <= 1'b0;
8          else q <= t ? ~q : q;
9      end
10 endmodule
11

```

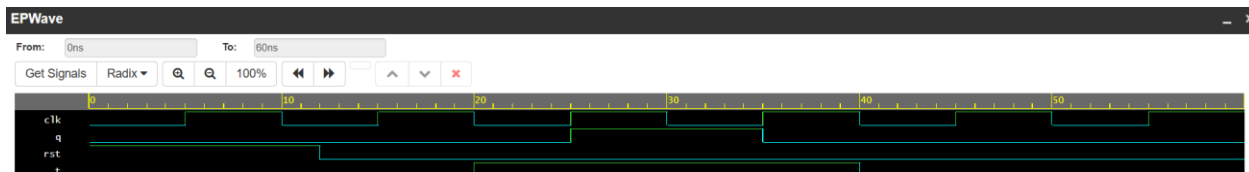
TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module tb_t_ff;
3      logic clk, rst, t, q;
4
5      t_ff uut (.clk(clk), .rst(rst), .t(t), .q(q));
6
7      always #5 clk = ~clk;
8
9      initial begin
10         $dumpfile("dump.vcd"); $dumpvars(0, tb_t_ff);
11
12         clk = 0; rst = 1; t = 0;
13         #12 rst = 0; // Release reset
14         #8 t = 1; // Toggles (q becomes 1)
15         #10 t = 1; // Toggles again (q becomes 0)
16         #10 t = 0; // Holds state (q stays 0)
17         #20 $finish;
18     end
19 endmodule

```

EP Wave:



SR-Flipflop:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module sr_ff (
3     input logic clk, rst, s, r,
4     output logic q
5 );
6     always_ff @(posedge clk or posedge rst) begin
7         if (rst) q <= 1'b0;
8         else begin
9             case ({s, r})
10                2'b01: q <= 1'b0; // Reset
11                2'b10: q <= 1'b1; // Set
12                2'b11: q <= 1'bx; // Invalid state
13                default: q <= q; // Hold (2'b00)
14            endcase
15        end
16    end
17 endmodule

```

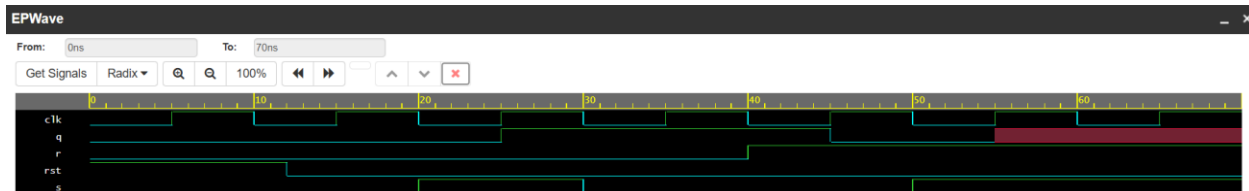
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module tb_sr_ff;
3     logic clk, rst, s, r, q;
4
5     sr_ff uut (.clk(clk), .rst(rst), .s(s), .r(r), .q(q));
6
7     always #5 clk = ~clk;
8
9     initial begin
10         $dumpfile("dump.vcd"); $dumpvars(0, tb_sr_ff);
11
12         clk = 0; rst = 1; s = 0; r = 0;
13         #12 rst = 0; // Release reset
14         #8 s = 1; r = 0; // Set (q becomes 1)
15         #10 s = 0; r = 0; // Hold (q stays 1)
16         #10 s = 0; r = 1; // Reset (q becomes 0)
17         #10 s = 1; r = 1; // Invalid state (q becomes X)
18         #20 $finish;
19     end
20 endmodule

```

EP Wave:



JK-Flipflop:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module jk_ff (
3     input logic clk, rst, j, k,
4     output logic q
5 );
6     always_ff @(posedge clk or posedge rst) begin
7         if (rst) q <= 1'b0;
8         else begin
9             case ({j, k})
10                2'b01: q <= 1'b0; // Reset
11                2'b10: q <= 1'b1; // Set
12                2'b11: q <= ~q; // Toggle
13                default: q <= q; // Hold (2'b00)
14            endcase
15        end
16    end
17 endmodule

```

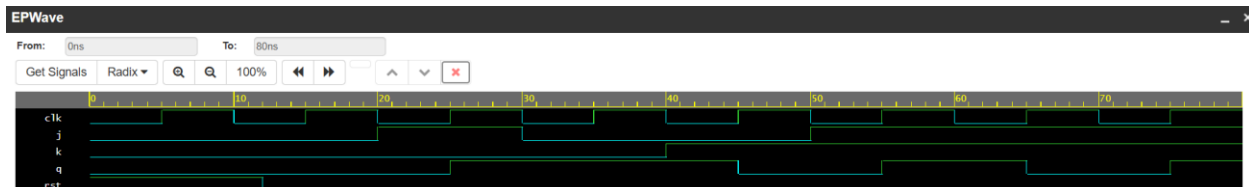
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module tb_jk_ff;
3     logic clk, rst, j, k, q;
4
5     jk_ff uut (.clk(clk), .rst(rst), .j(j), .k(k), .q(q));
6
7     always #5 clk = ~clk;
8
9     initial begin
10         $dumpfile("dump.vcd"); $dumpvars(0, tb_jk_ff);
11
12         clk = 0; rst = 1; j = 0; k = 0;
13         #12 rst = 0; // Release reset
14         #8 j = 1; k = 0; // Set (q becomes 1)
15         #10 j = 0; k = 0; // Hold (q stays 1)
16         #10 j = 0; k = 1; // Reset (q becomes 0)
17         #10 j = 1; k = 1; // Toggle (q becomes 1)
18         #10 j = 1; k = 1; // Toggle again (q becomes 0)
19         #20 $finish;
20     end
21 endmodule

```

EP Wave:



Conversion of T-FF into D-FF:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module t_ff (
3     input logic clk, rst, t,
4     output logic q
5 );
6     always_ff @(posedge clk or posedge rst) begin
7         if (rst) q <= 1'b0;
8         else q <= t ? ~q : q;
9     end
10 endmodule
11
12 module d_from_t_ff (
13     input logic clk, rst, d,
14     output logic q
15 );
16     logic t_input;
17
18     assign t_input = d ^ q;
19
20     t_ff internal_t_ff (
21         .clk(clk),
22         .rst(rst),
23         .t(t_input),
24         .q(q)
25     );
26 endmodule

```

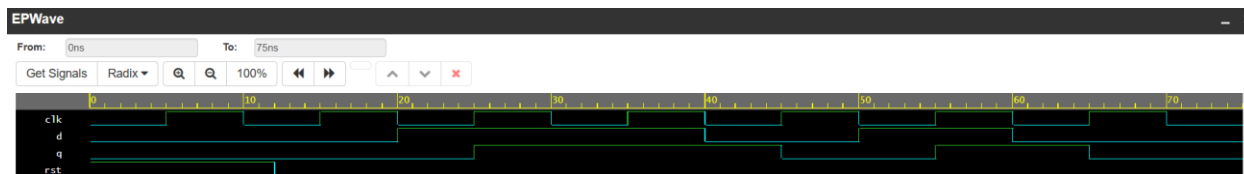
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module tb_d_from_t_ff;
3     logic clk, rst, d, q;
4
5     d_from_t_ff uut (
6         .clk(clk),
7         .rst(rst),
8         .d(d),
9         .q(q)
10    );
11
12    always #5 clk = ~clk;
13
14    initial begin
15        $dumpfile("dump.vcd");
16        $dumpvars(0, tb_d_from_t_ff);
17
18        clk = 0; rst = 1; d = 0;
19        #12 rst = 0;
20
21        // Drive D input variations
22        #8 d = 1; // Q should become 1
23        #10 d = 1; // Q should stay 1
24        #10 d = 0; // Q should become 0
25        #10 d = 1; // Q should become 1
26        #10 d = 0; // Q should become 0
27
28        #15 $finish;
29    end
30 endmodule

```

EP Wave:



Synchronous Up/Down Counter:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module sync_up_down_counter #(
3     parameter WIDTH = 4
4 );
5     input wire clk,
6     input wire rst_n,
7     // Active-low synchronous reset
8     input wire up_dn,
9     // 1 = Up, 0 = Down
10    input wire en,
11    output reg [WIDTH-1:0] count
12 );
13
14    always @(posedge clk) begin
15        if (!rst_n) begin
16            count <= {WIDTH{1'b0}};
17        end else if (en) begin
18            if (up_dn) begin
19                count <= count + 1'b1;
20            end else begin
21                count <= count - 1'b1;
22            end
23        end
24    end
25 endmodule

```

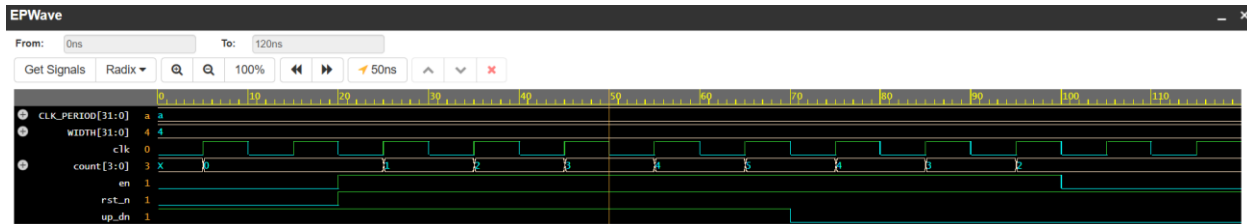
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module tb_sync_up_down_counter;
3
4     parameter WIDTH = 4;
5     parameter CLK_PERIOD = 10;
6
7     reg clk;
8     reg rst_n;
9     reg up_dn;
10    reg en;
11    wire [WIDTH-1:0] count;
12
13    sync_up_down_counter #(WIDTH(WIDTH))
14    dut (.clk(clk), .rst_n(rst_n), .up_dn(up_dn), .en(en),
15        .count(count));
16
17    initial begin
18        clk = 0;
19        forever #(CLK_PERIOD/2) clk = ~clk;
20    end
21
22    // Stimulus Block
23    initial begin
24        $dumpfile("dump.vcd");
25        $dumpvars(0, tb_sync_up_down_counter);
26
27        rst_n = 0;
28        up_dn = 1;
29        en = 0;
30        #(CLK_PERIOD * 2);
31
32        rst_n = 1;
33        en = 1;
34        #(CLK_PERIOD * 5);
35
36        up_dn = 0;
37        #(CLK_PERIOD * 3);
38
39        en = 0;
40        #(CLK_PERIOD * 2);
41
42        $finish;
43    end
44 endmodule

```

EP Wave:



Aynchronous Up/Down Counter:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module async_up_down_counter (
3      input logic clk,
4      input logic rst_n,
5      input logic up_dn,
6      output logic [3:0] count
7  );
8
9      logic [3:0] stage_clk;
10     logic [3:0] d_in;
11
12     assign stage_clk[0] = clk;
13     assign stage_clk[1] = up_dn ? ~count[0] : count[0];
14     assign stage_clk[2] = up_dn ? ~count[1] : count[1];
15     assign stage_clk[3] = up_dn ? ~count[2] : count[2];
16
17     assign d_in[0] = ~count[0];
18     assign d_in[1] = ~count[1];
19     assign d_in[2] = ~count[2];
20     assign d_in[3] = ~count[3];
21
22     always_ff @(posedge stage_clk[0] or negedge rst_n) begin
23         if (!rst_n) count[0] <= 1'b0;
24         else count[0] <= d_in[0];
25     end
26
27     always_ff @(posedge stage_clk[1] or negedge rst_n) begin
28         if (!rst_n) count[1] <= 1'b0;
29         else count[1] <= d_in[1];
30     end
31
32     always_ff @(posedge stage_clk[2] or negedge rst_n) begin
33         if (!rst_n) count[2] <= 1'b0;
34         else count[2] <= d_in[2];
35     end
36
37     always_ff @(posedge stage_clk[3] or negedge rst_n) begin
38         if (!rst_n) count[3] <= 1'b0;
39         else count[3] <= d_in[3];
40     end
41
42 endmodule

```

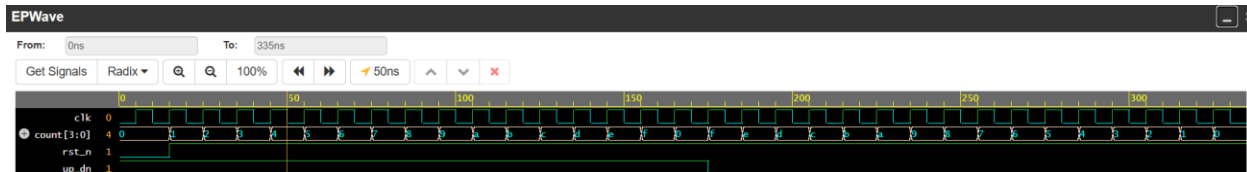
TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module tb_async_counter;
3
4      logic clk;
5      logic rst_n;
6      logic up_dn;
7      logic [3:0] count;
8
9      async_up_down_counter dut (
10         .clk(clk),
11         .rst_n(rst_n),
12         .up_dn(up_dn),
13         .count(count)
14     );
15
16     always #5 clk = ~clk;
17
18     initial begin
19         $dumpfile("dump.vcd");
20         $dumpvars(0, tb_async_counter);
21
22         clk = 0;
23         rst_n = 0;
24         up_dn = 1;
25         #15;
26
27         rst_n = 1;
28         #160;
29
30         up_dn = 0;
31         #160;
32
33         $finish;
34     end
35
36 endmodule

```

EP Wave:



BCD Counter:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module bcd_counter (
3      input logic      clk,
4      input logic      rst_n,
5      output logic [3:0] count
6  );
7
8      always_ff @(posedge clk or negedge rst_n) begin
9          if (!rst_n) begin
10             count <= 4'b0000;
11          end else begin
12             if (count == 4'd9)
13                 count <= 4'b0000;
14             else
15                 count <= count + 1'b1;
16             end
17         end
18     end
19 endmodule

```

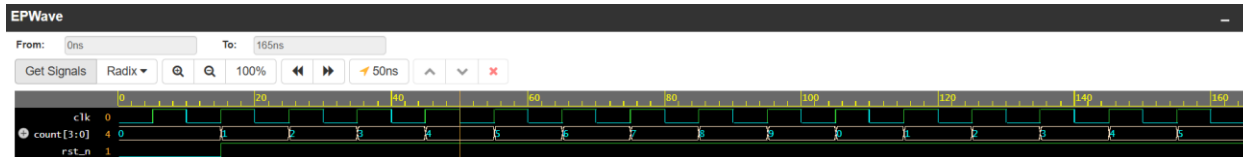
TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module tb_bcd_counter;
3
4      logic      clk;
5      logic      rst_n;
6      logic [3:0] count;
7
8      bcd_counter uut (
9          .clk(clk),
10         .rst_n(rst_n),
11         .count(count)
12     );
13
14     always #5 clk = ~clk;
15
16     initial begin
17         $dumpfile("dump.vcd");
18         $dumpvars(0, tb_bcd_counter);
19
20         clk = 0;
21         rst_n = 0;
22         #15;
23
24         rst_n = 1;
25         #150;
26
27         $finish;
28     end
29 endmodule

```

EP Wave:



Mod-N Counter:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module mod_n_counter #(
3      parameter N = 10,
4      parameter WIDTH = 4
5  )(
6      input logic      clk,
7      input logic      rst_n,
8      output logic [WIDTH-1:0] count
9  );
10
11     always_ff @(posedge clk or negedge rst_n) begin
12         if (!rst_n) begin
13             count <= 0;
14         end else begin
15             if (count == N - 1)
16                 count <= 0;
17             else
18                 count <= count + 1'b1;
19             end
20         end
21     end
22 endmodule

```

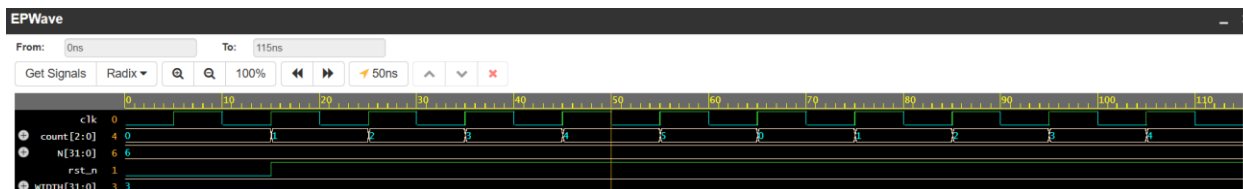
TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module tb_mod_n_counter;
3
4      parameter N = 6;
5      parameter WIDTH = 3;
6
7      logic      clk;
8      logic      rst_n;
9      logic [WIDTH-1:0] count;
10
11     mod_n_counter #(N(N), .WIDTH(WIDTH))
12     dut (.clk(clk), .rst_n(rst_n),
13         .count(count));
14
15     always #5 clk = ~clk;
16
17     initial begin
18         $dumpfile("dump.vcd");
19         $dumpvars(0, tb_mod_n_counter);
20
21         clk = 0;
22         rst_n = 0;
23         #15;
24
25         rst_n = 1;
26         #100;
27
28         $finish;
29     end
30 endmodule

```

EP Wave:



Frequency Divider 100Mhz-25Mhz:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module freq_divider (
3     input logic clk_in,
4     input logic rst_n,
5     output logic clk_out
6 );
7
8     logic qa;
9     logic qb;
10
11     always_ff @(negedge clk_in or negedge rst_n) begin
12         if (!rst_n)
13             qa <= 1'b0;
14         else
15             qa <= ~qa;
16     end
17
18     always_ff @(negedge qa or negedge rst_n) begin
19         if (!rst_n)
20             qb <= 1'b0;
21         else
22             qb <= ~qb;
23     end
24
25     assign clk_out = qb;
26
27 endmodule

```

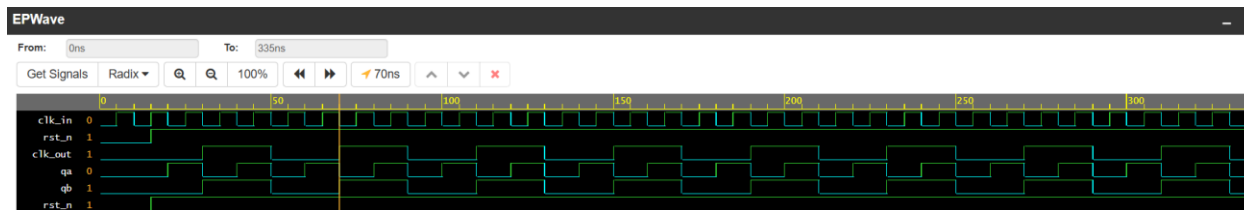
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module tb_freq_divider;
3
4     logic clk_in;
5     logic rst_n;
6     logic clk_out;
7
8     freq_divider uut (
9         .clk_in(clk_in),
10        .rst_n(rst_n),
11        .clk_out(clk_out)
12    );
13
14    always #5 clk_in = ~clk_in;
15
16    initial begin
17        $dumpfile("dump.vcd");
18        $dumpvars(0, tb_freq_divider);
19
20        clk_in = 0;
21        rst_n = 0;
22        #15;
23
24        rst_n = 1;
25        #320;
26
27        $finish;
28    end
29
30 endmodule

```

EP Wave:



PISO Register:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module piso (
3     input logic clk,
4     input logic load,
5     // 1 = Load, 0 = Shift
6     input logic [3:0] d_in,
7     output logic s_out
8 );
9
10    logic [3:0] shift_reg;
11
12    always_ff @(posedge clk) begin
13        if (load)
14            shift_reg <= d_in;
15        else
16            shift_reg <= {1'b0, shift_reg[3:1]};
17    end
18
19    assign s_out = shift_reg[0];
20
21 endmodule

```

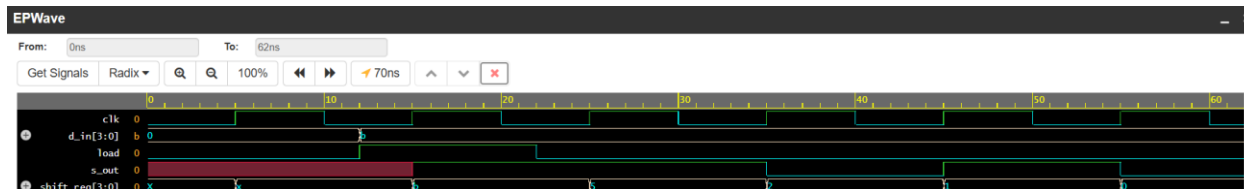
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module piso_tb;
3
4     logic clk, load;
5     logic [3:0] d_in;
6     logic s_out;
7
8     piso dut (.clk(clk), .load(load),
9             .d_in(d_in), .s_out(s_out));
10
11    always #5 clk = ~clk;
12
13    initial begin
14        $dumpfile("dump.vcd");
15        $dumpvars(0, piso_tb);
16
17        clk = 0;
18        load = 0;
19        d_in = 4'b0; #12;
20
21        load = 1;
22        d_in = 4'b1011; #10;
23
24        load = 0; #40;
25
26        $finish;
27    end
28
29 endmodule

```

EP Wave:



Bidirectional Shift Register:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module bidir_shift_reg (
3     input logic clk,
4     input logic rst_n,
5     input logic load,
6     input logic left_right,
7     // 1 = Left, 0 = Right
8     input logic [3:0] d_in,
9     output logic [3:0] q
10 );
11
12 always_ff @(posedge clk or negedge rst_n) begin
13     if (!rst_n)
14         q <= 4'b0000;
15     else if (load)
16         q <= d_in;
17     else if (left_right)
18         q <= {q[2:0], 1'b0}; // Shift Left
19     else
20         q <= {1'b0, q[3:1]}; // Shift Right
21 end
22 endmodule
23

```

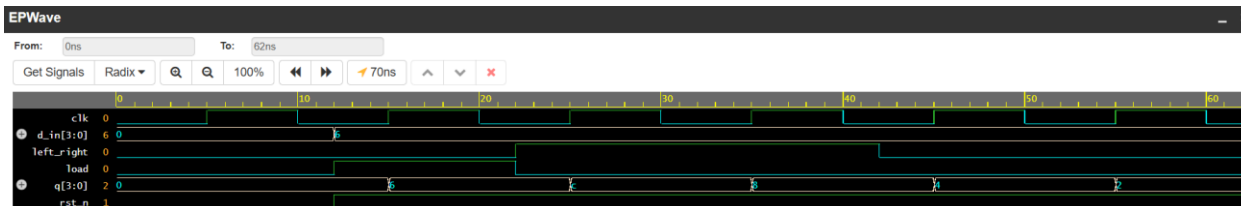
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module bidir_shift_reg_tb;
3
4     logic clk;
5     logic rst_n;
6     logic load;
7     logic left_right;
8     logic [3:0] d_in;
9     logic [3:0] q;
10
11     bidir_shift_reg uut (
12         .clk(clk),
13         .rst_n(rst_n),
14         .load(load),
15         .left_right(left_right),
16         .d_in(d_in),
17         .q(q)
18     );
19
20     always #5 clk = ~clk;
21
22     initial begin
23         $dumpfile("dump.vcd");
24         $dumpvars(0, bidir_shift_reg_tb);
25
26         clk = 0;
27         rst_n = 0;
28         load = 0;
29         left_right = 0;
30         d_in = 4'b0; #12;
31
32         rst_n = 1;
33         load = 1;
34         d_in = 4'b0110; #10;
35
36         load = 0;
37         left_right = 1; #20; // Shift Left
38
39         left_right = 0; #20; // Shift Right
40
41         $finish;
42     end
43 endmodule
44

```

EP Wave:



Universal Shift Register:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module universal_shift_reg (
3     input logic clk,
4     input logic rst_n,
5     input logic [1:0] mode,
6     input logic s_in_left,
7     input logic s_in_right,
8     input logic [3:0] d_in,
9     output logic [3:0] q
10 );
11
12 always_ff @(posedge clk or negedge rst_n) begin
13     if (!rst_n)
14         q <= 4'b0000;
15     else begin
16         case (mode)
17             2'b01: q <= {s_in_right, q[3:1]};
18             2'b10: q <= {q[2:0], s_in_left};
19             2'b11: q <= d_in;
20             default: q <= q;
21         endcase
22     end
23 end
24 endmodule
25

```

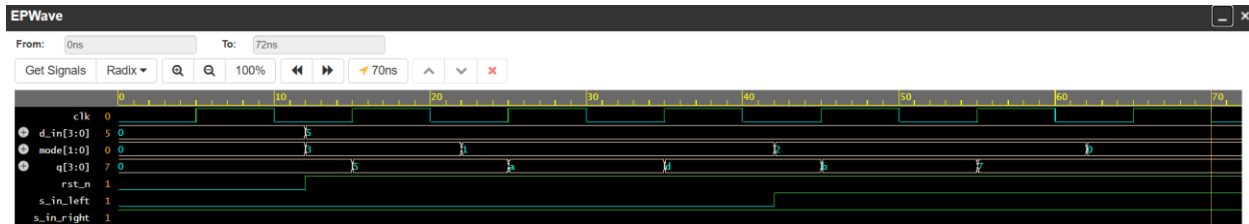
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module universal_shift_reg_tb;
3
4     logic clk;
5     logic rst_n;
6     logic [1:0] mode;
7     logic s_in_left;
8     logic s_in_right;
9     logic [3:0] d_in;
10    logic [3:0] q;
11
12    universal_shift_reg uut (
13        .clk(clk),
14        .rst_n(rst_n),
15        .mode(mode),
16        .s_in_left(s_in_left),
17        .s_in_right(s_in_right),
18        .d_in(d_in),
19        .q(q)
20    );
21
22    always #5 clk = ~clk;
23
24    initial begin
25        $dumpfile("dump.vcd");
26        $dumpvars(0, universal_shift_reg_tb);
27
28        clk = 0;
29        rst_n = 0;
30        mode = 2'b00;
31        s_in_left = 0;
32        s_in_right = 1;
33        d_in = 4'b0; #12;
34
35        rst_n = 1;
36        mode = 2'b11;
37        d_in = 4'b0101; #10;
38
39        mode = 2'b01; #20;
40
41        mode = 2'b10;
42        s_in_left = 1; #20;
43
44        mode = 2'b00; #10;
45
46        $finish;
47    end
48 endmodule
49

```


EP Wave:



Ring Counter:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module ring_counter (
3      input logic      clk,
4      input logic      rst_n,
5      output logic [3:0] q
6  );
7
8      always_ff @(posedge clk or negedge rst_n) begin
9          if (!rst_n)
10             q <= 4'b0001;
11          else
12             q <= {q[0], q[3:1]};
13      end
14  endmodule

```

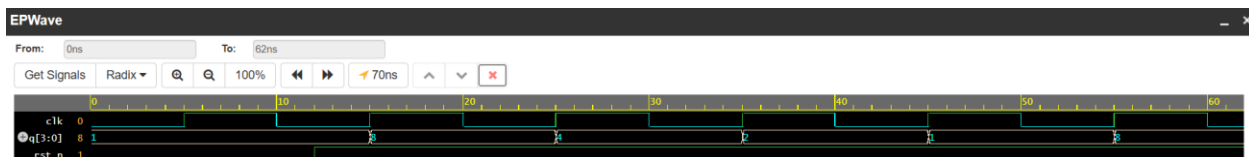
TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module ring_counter_tb;
3
4      logic      clk;
5      logic      rst_n;
6      logic [3:0] q;
7
8      ring_counter uut (
9          .clk(clk),
10         .rst_n(rst_n),
11         .q(q)
12     );
13
14     always #5 clk = ~clk;
15
16     initial begin
17         $dumpfile("dump.vcd");
18         $dumpvars(0, ring_counter_tb);
19
20         clk = 0;
21         rst_n = 0; #12;
22
23         rst_n = 1; #50;
24
25         $finish;
26     end
27 endmodule

```

EP Wave:



Jhonson Counter:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module johnson_counter (
3     input logic clk,
4     input logic rst_n,
5     output logic [3:0] q
6 );
7
8     always_ff @(posedge clk or negedge rst_n) begin
9         if (!rst_n)
10            q <= 4'b0000;
11        else
12            q <= {~q[0], q[3:1]};
13        end
14    endmodule

```

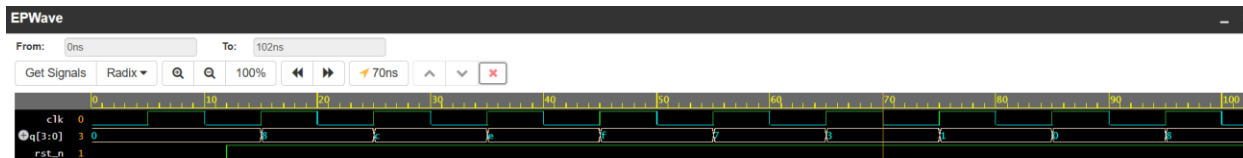
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module johnson_counter_tb;
3
4     logic clk;
5     logic rst_n;
6     logic [3:0] q;
7
8     johnson_counter uut (
9         .clk(clk),
10        .rst_n(rst_n),
11        .q(q)
12    );
13
14    always #5 clk = ~clk;
15
16    initial begin
17        $dumpfile("dump.vcd");
18        $dumpvars(0, johnson_counter_tb);
19
20        clk = 0;
21        rst_n = 0; #12;
22
23        rst_n = 1; #90;
24
25        $finish;
26    end
27 endmodule

```

EP Wave:



Multiplier using Pipelining:

Design:

```

1 /*****Abubakar Atique DUET*****/
2 module pipelined_multiplier (
3     input clk,
4     input rst,
5     input [31:0] a,
6     input [31:0] b,
7     output reg [63:0] product
8 );
9     reg [31:0] a_reg, b_reg;
10    reg [63:0] mult_reg;
11
12    always @(posedge clk or posedge rst) begin
13        if (rst) begin
14            a_reg <= 32'd0;
15            b_reg <= 32'd0;
16            mult_reg <= 64'd0;
17            product <= 64'd0;
18        end else begin
19            a_reg <= a;
20            b_reg <= b;
21
22            mult_reg <= a_reg * b_reg;
23
24            product <= mult_reg;
25        end
26    end
27 endmodule

```

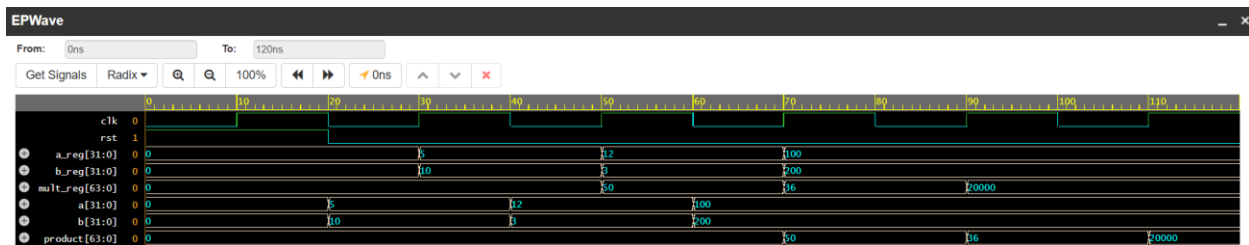
TestBench:

```

1 /*****Abubakar Atique DUET*****/
2 module tb_pipelined_multiplier;
3     reg clk;
4     reg rst;
5     reg [31:0] a;
6     reg [31:0] b;
7     wire [63:0] product;
8
9     pipelined_multiplier dut (
10        .clk(clk), .rst(rst), .a(a), .b(b),
11        .product(product) );
12
13    always #10 clk = ~clk;
14
15    initial begin
16        $dumpfile("dump.vcd");
17        $dumpvars(0, tb_pipelined_multiplier);
18
19        clk = 0;
20        rst = 1;
21        a = 0;
22        b = 0;
23
24        #20; rst = 0;
25
26        a = 32'd5;
27        b = 32'd10; #20;
28
29        a = 32'd12;
30        b = 32'd3; #20;
31
32        a = 32'd100;
33        b = 32'd200; #20;
34
35        #40; $finish;
36    end
37 endmodule

```

EP Wave:



ALU using Pipelining:

Design:

```

1  /*****Abubakar Atique DUET*****/
2  module pipelined_alu (
3      input clk,
4      input rst,
5      input [2:0] sel,
6      input [31:0] a,
7      input [31:0] b,
8      output reg [63:0] result
9  );
10     reg [2:0] sel_reg1;
11     reg [31:0] a_reg1, b_reg1;
12
13     reg [63:0] alu_reg2;
14
15     always @(posedge clk or posedge rst) begin
16         if (rst) begin
17             sel_reg1 <= 3'd0;
18             a_reg1 <= 32'd0;
19             b_reg1 <= 32'd0;
20             alu_reg2 <= 64'd0;
21             result <= 64'd0;
22         end else begin
23             sel_reg1 <= sel;
24             a_reg1 <= a;
25             b_reg1 <= b;
26
27             case (sel_reg1)
28                 3'b000: alu_reg2 <= a_reg1 + b_reg1;
29                 3'b001: alu_reg2 <= a_reg1 - b_reg1;
30                 3'b010: alu_reg2 <= a_reg1 * b_reg1;
31                 3'b011: alu_reg2 <= a_reg1 & b_reg1;
32                 3'b100: alu_reg2 <= a_reg1 | b_reg1;
33                 3'b101: alu_reg2 <= a_reg1 ^ b_reg1;
34                 3'b110: alu_reg2 <= ~a_reg1;
35                 3'b111: alu_reg2 <= (a_reg1 > b_reg1)
36                     ? 64'd1 : 64'd0;
37             default: alu_reg2 <= 64'd0;
38             endcase
39
40             result <= alu_reg2;
41         end
42     end
43 endmodule

```

TestBench:

```

1  /*****Abubakar Atique DUET*****/
2  module tb_pipelined_alu;
3      reg clk;
4      reg rst;
5      reg [2:0] sel;
6      reg [31:0] a;
7      reg [31:0] b;
8      wire [63:0] result;
9
10     pipelined_alu dut (
11         .clk(clk),
12         .rst(rst),
13         .sel(sel),
14         .a(a),
15         .b(b),
16         .result(result)
17     );
18
19     always #10 clk = ~clk;
20
21     initial begin
22         $dumpfile("dump.vcd");
23         $dumpvars(0, tb_pipelined_alu);
24
25         clk = 0;
26         rst = 1;
27         sel = 0;
28         a = 0;
29         b = 0;
30
31         #20; rst = 0;
32
33         sel = 3'b000; a = 32'd25; b = 32'd5; #20;
34
35         sel = 3'b010; a = 32'd10; b = 32'd4; #20;
36
37         sel = 3'b111; a = 32'd50; b = 32'd20; #20;
38
39         sel = 3'b111; a = 32'd15; b = 32'd40; #20;
40
41         #40; $finish;
42     end
43 endmodule

```

EP Wave:

